National Higher School of Advanced Technologies

Academic year: 2026-2027 | First-year master's degree

Machine learning - Practical session 3

Hypothesis spaces, feature representations and model generalization

Objective

Explore how different machine learning models represent relationships in data, investigate how model complexity and feature representation affect learning performance, and evaluate the ability of models to generalize to unseen observations.

Exercise: Study hours and exam scores

Consider the following dataset containing the number of study hours and the corresponding exam scores of ten students.

Student	Study Hours (x)	Exam Score (y)
S1	1	42
S2	2	48
S3	3	55
S4	4	59
S5	5	66
S6	6	70
S7	7	78
S8	8	81
S9	9	89
S10	10	91

The objective is to predict students' exam scores based on their study hours using different regression models.

Part A : Exploring different hypothesis spaces

1. Explore the dataset

- Load the dataset into a Pandas DataFrame.

- Display its dimensions and descriptive statistics.
- Represent the relationship between study hours and exam scores using a scatter plot. **2.**

Fit different regression models

Use predefined Scikit-learn functions to train the following models:

- Linear Regression.
- Polynomial Regression (degree 2).
- Polynomial Regression (degree 3).
- Decision Tree (Maximum depth = 1)
- Decision Tree (Maximum depth = 3)
- K-Nearest Neighbors Regression

For polynomial regression, use PolynomialFeatures to transform the input features before applying Linear Regression.

3. Compare the models

- Plot the predictions of all six models together with the original observations. Compare their fitted curves and observe how different hypothesis spaces represent the relationship between study hours and exam scores.
- Evaluate and compare their training performance using RMSE and R^2.

Record your results in the following table:

Model	**Training RMSE**	**Training R^2**
Linear Regression		
Polynomial Regression (degree 2)		
Polynomial Regression (degree 3)		
Decision Tree (depth 1)		
Decision Tree (depth 3)		
KNN (k = 3)		

Part B : Investigating model complexity and generalization

Increasing model complexity can improve the fit to training observations, but it does not necessarily

improve predictions on unseen data.

1. Prepare the training and test datasets using Leave-One-Out

Since the dataset contains only ten observations, use Leave-One-Out Cross-Validation (LOOCV).

Use the predefined Scikit-learn function:

- `LeaveOneOut()`: Generates training and test indices.

Tasks:

- Initialize `LeaveOneOut()`.
- Generate and display the training and test sets for each iteration.
- Observe how the datasets change across iterations.

Note: LOOCV performs 10 iterations. Each iteration uses 9 observations for training and 1 observation for testing.

2. Explore different polynomial degrees

Train polynomial regression models using the following degrees:

- Degree 1
- Degree 2
- Degree 3
- Degree 5
- Degree 9

Use the same dataset for all experiments.

3. Evaluate generalization

Since the dataset contains only ten observations, use Leave-One-Out Cross-Validation (LOOCV) to evaluate the models.

Use predefined Scikit-learn functions:

- LeaveOneOut()
- cross_val_score()

For each polynomial degree, calculate the Training RMSE by fitting the model on the complete dataset. Calculate the LOOCV RMSE using cross_val_score() with LeaveOneOut().

4. Analyze the results

Complete the following table:

Polynomial Degree	Training RMSE	LOOCV RMSE
1		
2		
3		
5		
9		

- Compare training RMSE and LOOCV RMSE.
- Identify the model with the lowest LOOCV RMSE.
- Observe how model complexity affects training performance and generalization. •

Plot the LOOCV RMSE against polynomial degree.

Part C : Investigating feature representation and regularization

A machine learning model learns from the features used to represent the data. Different feature representations can influence its ability to capture relationships and generalize.

1. Compare feature representations

Use the following representations:

- Original representation: study hours (x).
- Polynomial representation: x, x^2 and x^3.
- Higher-degree polynomial representation: x, x^2, ..., x^5.

Compare the predictive performance of the resulting models using LOOCV.

2. Investigate regularization

Apply two regularized regression models to the polynomial features:

- Ridge Regression (L2 regularization).
- Lasso Regression (L1 regularization).

Use the predefined Scikit-learn implementations:

- Ridge()
- Lasso()

Use a Scikit-learn Pipeline to combine feature transformation, feature scaling and regression. This ensures that preprocessing is performed correctly within each cross-validation fold.

Test a few regularization strengths (alpha), for example:

- 0.01
- 0.1
- 1
- 10

Evaluate the models using LOOCV RMSE.

Then investigate Ridge and Lasso using degree-5 polynomial features.

3. Compare the results

Complete the following table:

Model	**Feature Degree**	**Regularization**	**LOOCV RMSE**
Linear Regression	1	None	
Polynomial Regression	3	None	
Polynomial Regression	5	None	
Ridge Regression	5	L2	
Lasso Regression	5	L1	
Decision Tree	depth 1	None	
Decision Tree	depth 3	None	
KNN	k = 3	None	

Identify whether regularization improves the predictive performance of polynomial regression. **Part D : Final model comparison**

Select the best configuration of each model family based on the experiments conducted in Parts A, B and C. Compare their training and LOOCV RMSE using the same evaluation strategy.

Your comparison should include:

- Linear Regression.
- Polynomial Regression.
- Ridge Regression.
- Lasso Regression.
- Decision Tree
- KNN

For each model:

1. Report the training RMSE.
2. Report the LOOCV RMSE.
3. Compare the fitted predictions with the original observations.
4. Identify the model that provides the best balance between fitting the available data and generalizing to unseen observations.

Present your final results in a summary table and a comparative plot.

Deliverables

Submit a Python code containing:

1. Dataset exploration.
2. Implementation using predefined Scikit-learn functions.
3. Training and cross-validation results.
4. Comparative plots.
5. Final summary table.
6. A short conclusion discussing the influence of model complexity, feature representation and regularization on generalization performance.

Important: All models must be evaluated using the same dataset and the same cross-validation strategy. For preprocessing operations such as feature scaling, use a Pipeline to avoid data leakage.

Model	Scikit-learn function / class	Configuration
Linear Regression	LinearRegression()	Default parameters
Polynomial Regression	PolynomialFeatures() + LinearRegression()	Degree = 2
Polynomial Regression	PolynomialFeatures() + LinearRegression()	Degree = 3

Decision Tree Regression	DecisionTreeRegressor()	Maximum depth = 1
Decision Tree Regression	DecisionTreeRegressor()	Maximum depth = 3
K-Nearest Neighbors Regression	KNeighborsRegressor()	Number of neighbors (k) = 3
Ridge Regression	Ridge()	L2 regularization; alpha ∈ 10
Lasso Regression	Lasso()	L1 regularization; alpha ∈ 10
Leave-One-Out Cross Validation (LOOCV)	LeaveOneOut()	10 folds
Cross-Validation Evaluation	cross_val_score()	Scoring = neg_mean_squared_error
Feature Scaling	StandardScaler()	Standardization (mean = 0, standard deviation = 1)